```
pip install openpyxl
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting et-xmlfile (from openpyxl)
  Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et-xmlfile, openpyxl
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
Note: you may need to restart the kernel to use updated packages.
```

```
import pandas as pd

# Load file
file_path = "Suggestions YTD 6-11.xlsx"
df = pd.read_excel(file_path, sheet_name="in")
```

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 154702 entries, 0 to 154701
Data columns (total 12 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   Record Type                  154702 non-null  object
 1   Posted Date                  154702 non-null  datetime64[ns]
 2   Expiration Date              154702 non-null  datetime64[ns]
 3   Actioned                     99983 non-null   float64
 4   Dismissed                    27143 non-null   float64
 5   Marked As Complete           7987 non-null    float64
 6   Title                        154702 non-null  object
 7   Reason                       154702 non-null  object
 8   Last Modified Date           154702 non-null  datetime64[ns]
 9   Last Modified By: Full Name  154702 non-null  object
 10  Owner: Full Name             154702 non-null  object
 11  Account: Name                154702 non-null  object
dtypes: datetime64[ns](3), float64(3), object(6)
memory usage: 14.2+ MB
```

```
# Clean HTML from the Reason Field
from bs4 import BeautifulSoup

df['Reason'] = df['Reason'].apply(lambda x: BeautifulSoup(x, "html.parser").get_text()
if isinstance(x, str) else "")
```

```
df['Actioned'] = df['Actioned'].fillna(0)
df['Dismissed'] = df['Dismissed'].fillna(0)
df['Marked As Complete'] = df['Marked As Complete'].fillna(0)
```

Advanced NLP

```python
# Data Prepration
# Clean title
df['Title'] = df['Title'].astype(str).str.lower().str.strip()

# Combine Title and Reason
df['Combined_Text'] = (df['Title'] + " " + df['Reason']).str.lower().str.strip()

# Drop any blank records
text_corpus = df['Combined_Text'].dropna().loc[lambda x: x.str.strip() != '']

# Preview
text_corpus.head(10)
```

```
0    high cgm growth adriana cadena-martinez's cgm ...
1    leading fsl trx growth with respect to goal ad...
2    hcp dexcom adoption for conventional insulin (...
3    lagging product transition alberto almeida's f...
4    hcp scripts are not being filled alberto almei...
5    low mdi patient total change alberto almeida?s...
6    hcp dexcom adoption for conventional insulin (...
7    coverage quality change aggregated (commercial...
8    lagging fsl trx growth with respect to goal al...
9    hcp dexcom adoption for oral patients alfredo ...
Name: Combined_Text, dtype: object
```

```python
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import numpy as np
from bs4 import BeautifulSoup
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.manifold import TSNE
from sklearn.decomposition import LatentDirichletAllocation
```

```python
# Step 1: Bigram Extraction for Theme Mapping

# Initialize CountVectorizer for bigrams
vectorizer = CountVectorizer(ngram_range=(2, 2), stop_words='english')
X = vectorizer.fit_transform(text_corpus)

# Sum up the counts of each bigram
bigram_counts = X.sum(axis=0).A1
bigram_vocab = vectorizer.get_feature_names_out()
bigram_freq = pd.DataFrame({'bigram': bigram_vocab, 'count': bigram_counts})
bigram_freq = bigram_freq.sort_values(by='count', ascending=False).head(30)

# Preview
bigram_freq.head(10)
```

	bigram	count
4086	13 weeks	134948
218304	visit account	127268
68623	fsl trx	110565
215212	understand driver	98006
221029	weeks schedule	70764
12414	account 2025	70717
167699	preceding 13	66228
214343	trx share	53700
193785	sell fsl	53184
221020	weeks compared	51662

```
# Step 2: Visualize Top Bigrams as a Treemap

# Step 1: Extract bigrams
vectorizer = CountVectorizer(ngram_range=(2, 2), stop_words='english')
X = vectorizer.fit_transform(text_corpus)

# Step 2: Create frequency dataframe
bigram_counts = X.sum(axis=0).A1
bigram_vocab = vectorizer.get_feature_names_out()
bigram_freq = pd.DataFrame({'bigram': bigram_vocab, 'count': bigram_counts})
bigram_freq = bigram_freq.sort_values(by='count', ascending=False).head(30)

# Step 3: Treemap plot
fig_treemap = px.treemap(bigram_freq,
                         path=['bigram'],
                         values='count',
                         title='Top 30 Frequent Bigrams from Suggestions (Title + Reaso
n)')

fig_treemap.update_layout(margin=dict(t=50, l=25, r=25, b=25))
fig_treemap.show()

# Step 4: Add accompanying table
fig_table = go.Figure(data=[go.Table(
    header=dict(values=["Bigram", "Frequency"],
                fill_color='paleturquoise',
                align='left'),
    cells=dict(values=[bigram_freq['bigram'], bigram_freq['count']],
               fill_color='lavender',
               align='left'))
])

fig_table.update_layout(title="Bigram Frequency Table (Top 30)")
fig_table.show()
```

Top 30 Frequent Bigrams from Suggestions (Title + Reason)

Bigram Frequency Table (Top 30)

Bigram	Frequency
13 weeks	134948
visit account	127268
fsl trx	110565
understand driver	98006
weeks schedule	70764
account 2025	70717
preceding 13	66228
trx share	53700
sell fsl	53184
weeks compared	51662
patients claims	51651
claims data	51651

LDA Topic Modelling

```python
# Cell 1: Imports
import pandas as pd
from bs4 import BeautifulSoup
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation
from collections import Counter
import plotly.express as px

# Cell 2: Load & (optionally) Sample Data
FPATH = "Suggestions YTD 6-11.xlsx"
SHEET = "in"
SAMPLE_SIZE = 10000  # set to None to use full dataset

df = pd.read_excel(FPATH, sheet_name=SHEET)
if SAMPLE_SIZE:
    df = df.sample(n=SAMPLE_SIZE, random_state=42)


# Cell 3: Clean & Combine Text
df['Title'] = df['Title'].astype(str).str.lower().str.strip()
df['reason_text'] = (
    df['Reason'].fillna('')
                 .apply(lambda x: BeautifulSoup(x, 'html.parser').get_text())
                 .str.lower()
                 .str.strip()
)
df['combined_text'] = (df['Title'] + ' ' + df['reason_text']).str.strip()
df = df[df['combined_text'].str.replace(r'\s+', '', regex=True) != '']

# Cell 4: Vectorize to DTM
vectorizer = CountVectorizer(
    stop_words='english',
    max_features=1_000,
    ngram_range=(1, 2)
)
dtm = vectorizer.fit_transform(df['combined_text'])

# Cell 5: Fit LDA
N_TOPICS = 6
lda = LatentDirichletAllocation(
    n_components=N_TOPICS,
    random_state=42,
    learning_method='batch'
)
lda.fit(dtm)

# Cell 6: Extract Keywords & Auto-Label
def get_topic_keywords(lda_model, vect, top_n=10):
    terms = vect.get_feature_names_out()
    kw = {}
```

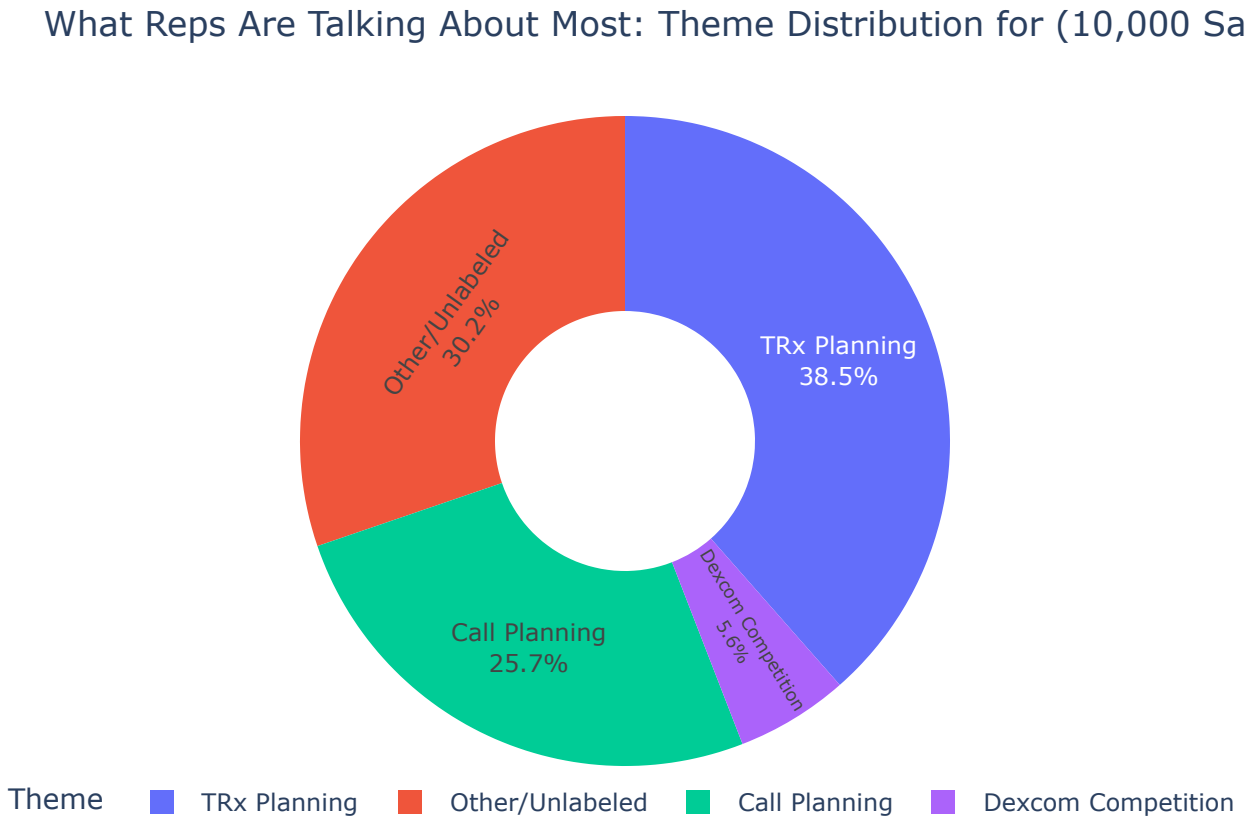
```
    for i, comp in enumerate(lda_model.components_):
        top_idx = comp.argsort()[-top_n:][::-1]
        kw[f"Topic {i+1}"] = [terms[j] for j in top_idx]
    return kw

topic_keywords = get_topic_keywords(lda, vectorizer)

def auto_label_topics(keywords):
    mapping = {
        "dexcom": "Dexcom Competition",
        "coverage": "Coverage & Access",
        "access": "Coverage & Access",
        "salesassist": "SalesAssist Prompts",
        "trx": "TRx Planning",
        "schedule": "Call Planning",
    }
    labels = {}
    for idx, (_, words) in enumerate(keywords.items()):
        hits = [mapping[w] for w in words if w in mapping]
        labels[idx] = Counter(hits).most_common(1)[0][0] if hits else "Other/Unlabeled"
    return labels

topic_labels = auto_label_topics(topic_keywords)
```

```
 # Cell 7: Assign Dominant Topic to Each Row
topic_dist = lda.transform(dtm)
df['dominant_topic'] = topic_dist.argmax(axis=1)
df['topic_label'] = df['dominant_topic'].map(topic_labels)

# Cell 8: Sanity-Check: print 3 examples per topic
for idx, label in topic_labels.items():
    print(f"\n--- Topic {idx+1}: {label} ---")
    samples = df[df['dominant_topic']==idx]['combined_text'].head(3)
    for s in samples:
        print("-", s[:200], "…")
```

```
--- Topic 1: Other/Unlabeled ---
- lifestyle outcomes fsl portfolio in your last call with adetokunbo akinyeye on 2025-0
- hcp dexcom adoption for multiple daily injections (mdi) patients crystal brown?s fsl
- suggested action: plan your next call your hcp, kathy nguyen casado, has not received

--- Topic 2: Dexcom Competition ---
- dexcom trx growth with respect to similar hcp john reynolds's dexcom trx grew 39.7% i
- access: accessibility/affordable a crm email, tir correlation with a1c, was recently
- dexcom trx growth with respect to similar hcp stanley scheidler's dexcom trx grew 70.

--- Topic 3: TRx Planning ---
- high cgm growth shanta hopson's cgm trx has increased 41.0% in the last 13 weeks (64)
- hcp dexcom adoption for multiple daily injections (mdi) patients jeremy grogg?s fsl t
- lagging fsl trx growth with respect to goal syeda naqvi's fsl prescriptions are track

--- Topic 4: Other/Unlabeled ---
- suggested action: plan your next call your hcp, michelle midkiff, has not received a
```

- cgm industry guidelines rte, cgm selective hcp in your last call with gerald krumpos

```
# Cell 9: Business-Ready Pie Chart
# - show absolute counts and percentages in a donut-style pie

# prepare summary
summary = (
    df['topic_label']
      .value_counts()
      .rename_axis('topic_label')
      .reset_index(name='count')
)

# plot
fig = px.pie(
    summary,
    names='topic_label',
    values='count',
    title=f"What Reps Are Talking About Most: Theme Distribution for ({len(df):,} Sampl
es)",
    hole=0.4         # makes it a donut
)

# show both percent + label inside each slice
fig.update_traces(
    textposition='inside',
    textinfo='percent+label',
    showlegend=True
)

# tweak layout for presentation
fig.update_layout(
    margin=dict(l=40, r=40, t=80, b=40),
    legend_title_text='Theme',
    legend=dict(orientation='h', yanchor='bottom', y=-0.1, xanchor='center', x=0.5)
)

fig.show()
```

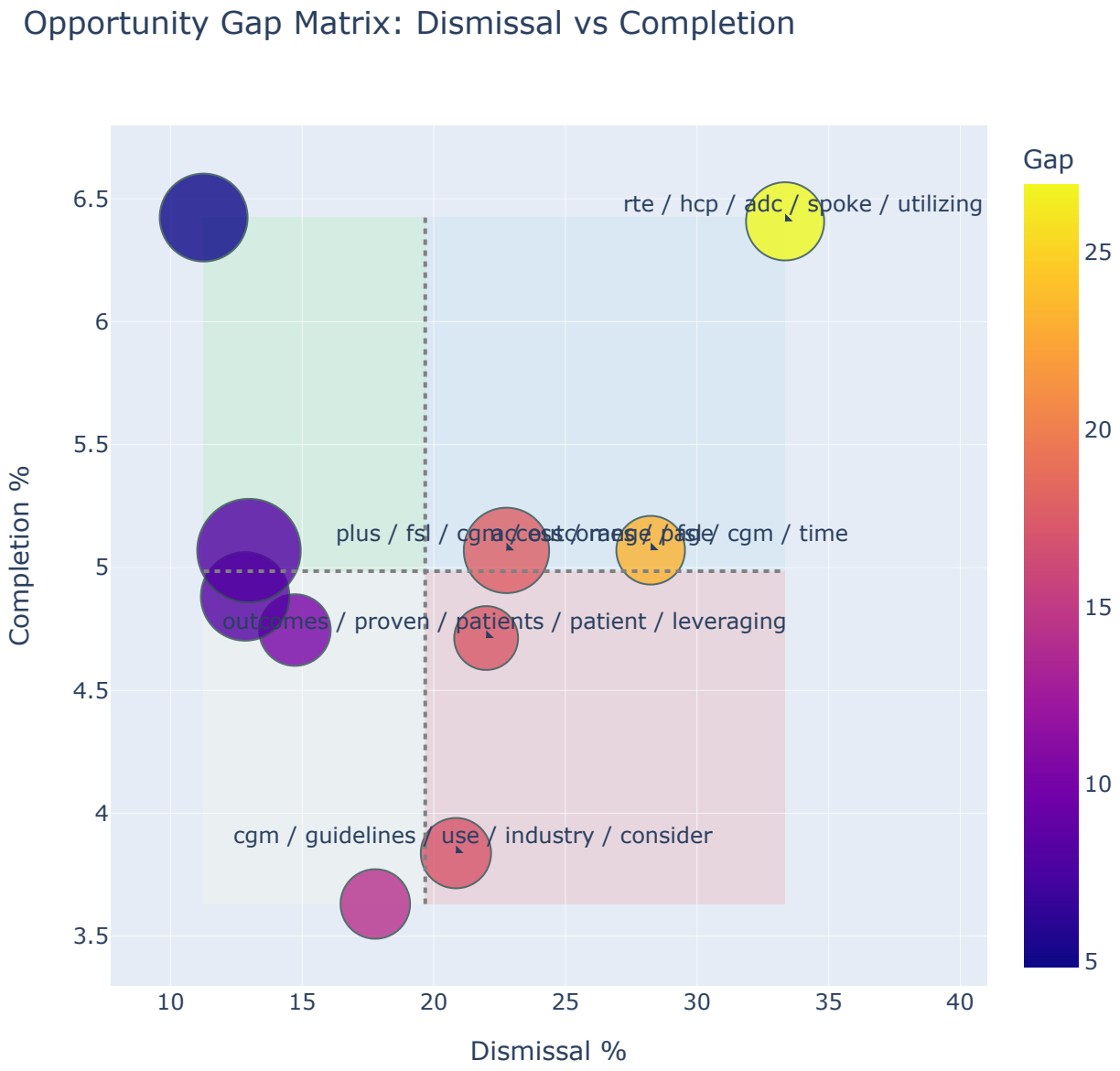


Opportunity Gap Matrix

```
import pandas as pd
import numpy as np
from bs4 import BeautifulSoup
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation
import plotly.express as px

# === 1. Load & clean ===
file_path = "Suggestions YTD 6-11.xlsx"
df = pd.read_excel(file_path, sheet_name="in")

# Clean Title & Reason, build combined_text
df['Title'] = df['Title'].astype(str).str.lower().str.strip()
df['reason_text'] = (
    df['Reason'].fillna('')
      .apply(lambda x: BeautifulSoup(x, 'html.parser').get_text())
      .str.lower().str.strip()
)
df['combined_text'] = (
    df['Title'] + " " + df['reason_text']
).str.replace(r'\s+', ' ', regex=True).str.strip()
df = df[df['combined_text'] != '']  # drop blanks

# === 2. Vectorize & fit LDA ===
```

```
cv = CountVectorizer(max_features=5000, stop_words="english")
dtm = cv.fit_transform(df['combined_text'])

n_topics = 10  # adjust as needed
lda = LatentDirichletAllocation(n_components=n_topics, random_state=42)
lda.fit(dtm)

# === 3. Assign each row to its top topic ===
topic_probs = lda.transform(dtm)
df['Topic'] = topic_probs.argmax(axis=1)

# Optional: name your topics based on top words
def get_topic_labels(model, vectorizer, n_top=5):
    labels = {}
    vocab = vectorizer.get_feature_names_out()
    for idx, comp in enumerate(model.components_):
        terms = [vocab[i] for i in comp.argsort()[-n_top:]]
        labels[idx] = " / ".join(terms[::-1])
    return labels

topic_map = get_topic_labels(lda, cv, n_top=5)
df['Topic_Label'] = df['Topic'].map(topic_map)

# === 4. Compute Opportunity Gap metrics ===
for col in ['Dismissed', 'Marked As Complete', 'Actioned']:
    df[col] = pd.to_numeric(df[col].fillna(0), errors='coerce')

df_gap = df.dropna(subset=['Topic_Label']).copy()
theme_summary = (
    df_gap.groupby('Topic_Label')
          .agg(
            Total_Suggestions=('Topic_Label','count'),
            Num_Completed    =('Marked As Complete','sum'),
            Num_Dismissed    =('Dismissed','sum'),
            Num_Actioned     =('Actioned','sum')
          )
          .reset_index()
)
theme_summary['% Completed'] = 100 * theme_summary['Num_Completed'] / theme_summary['To
tal_Suggestions']
theme_summary['% Dismissed'] = 100 * theme_summary['Num_Dismissed'] / theme_summary['To
tal_Suggestions']
theme_summary['% Actioned']  = 100 * theme_summary['Num_Actioned']  / theme_summary['To
tal_Suggestions']
theme_summary['Gap']         = theme_summary['% Dismissed'] -
 theme_summary['% Completed']
theme_summary['Size']        = np.sqrt(theme_summary['Total_Suggestions'])

# === 5. Plot the Opportunity Gap Matrix ===
mean_dismiss  = theme_summary['% Dismissed'].mean()
mean_complete = theme_summary['% Completed'].mean()
xmin, xmax = theme_summary['% Dismissed'].min(), theme_summary['% Dismissed'].max()
ymin, ymax = theme_summary['% Completed'].min(), theme_summary['% Completed'].max()
```

```
fig = px.scatter(
    theme_summary,
    x='% Dismissed', y='% Completed',
    size='Size', color='Gap',
    hover_data=['Total_Suggestions','% Actioned','Gap'],
    title='Opportunity Gap Matrix: Dismissal vs Completion',
    labels={'% Dismissed':'Dismissal %','% Completed':'Completion %'},
    size_max=40
)

# quadrant lines
fig.add_shape(type='line', x0=mean_dismiss, x1=mean_dismiss, y0=ymin, y1=ymax,
              line=dict(color='gray', dash='dot'))
fig.add_shape(type='line', x0=xmin, x1=xmax, y0=mean_complete, y1=mean_complete,
              line=dict(color='gray', dash='dot'))

# quadrant shading
fig.add_shape(type='rect', x0=xmin, x1=mean_dismiss, y0=mean_complete, y1=ymax,
              fillcolor='lightgreen', opacity=0.2, layer='below', line_width=0)
fig.add_shape(type='rect', x0=mean_dismiss, x1=xmax, y0=mean_complete, y1=ymax,
              fillcolor='lightblue', opacity=0.2, layer='below', line_width=0)
fig.add_shape(type='rect', x0=mean_dismiss, x1=xmax, y0=ymin, y1=mean_complete,
              fillcolor='lightcoral', opacity=0.2, layer='below', line_width=0)
fig.add_shape(type='rect', x0=xmin, x1=mean_dismiss, y0=ymin, y1=mean_complete,
              fillcolor='lightyellow', opacity=0.2, layer='below', line_width=0)

# annotate top 5 gaps
top5 = theme_summary.nlargest(5, 'Gap')
for _, r in top5.iterrows():
    fig.add_annotation(x=r['% Dismissed'], y=r['% Completed'],
                       text=r['Topic_Label'], showarrow=True,
                       arrowhead=1, ax=10, ay=-10)

fig.update_traces(marker=dict(opacity=0.8, line=dict(width=1, color='DarkSlateGrey')))
fig.update_layout(height=650, legend_title='Gap (Dismissed - Completed)')
fig.show()
```

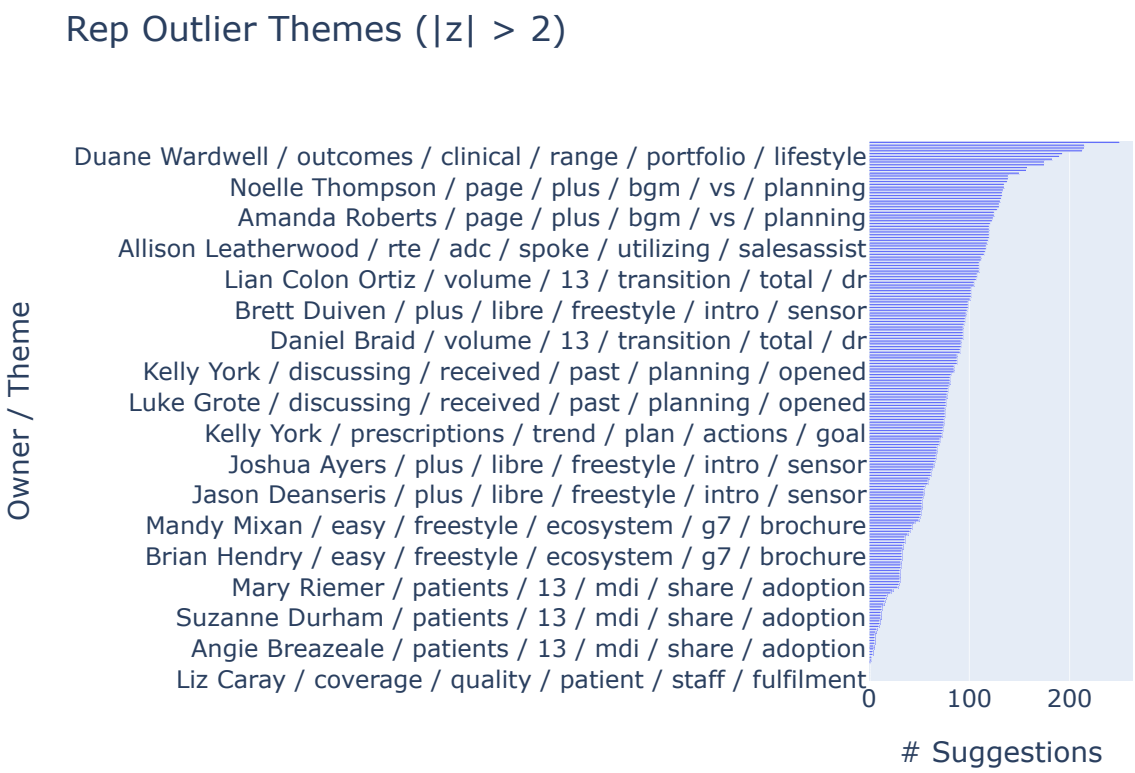


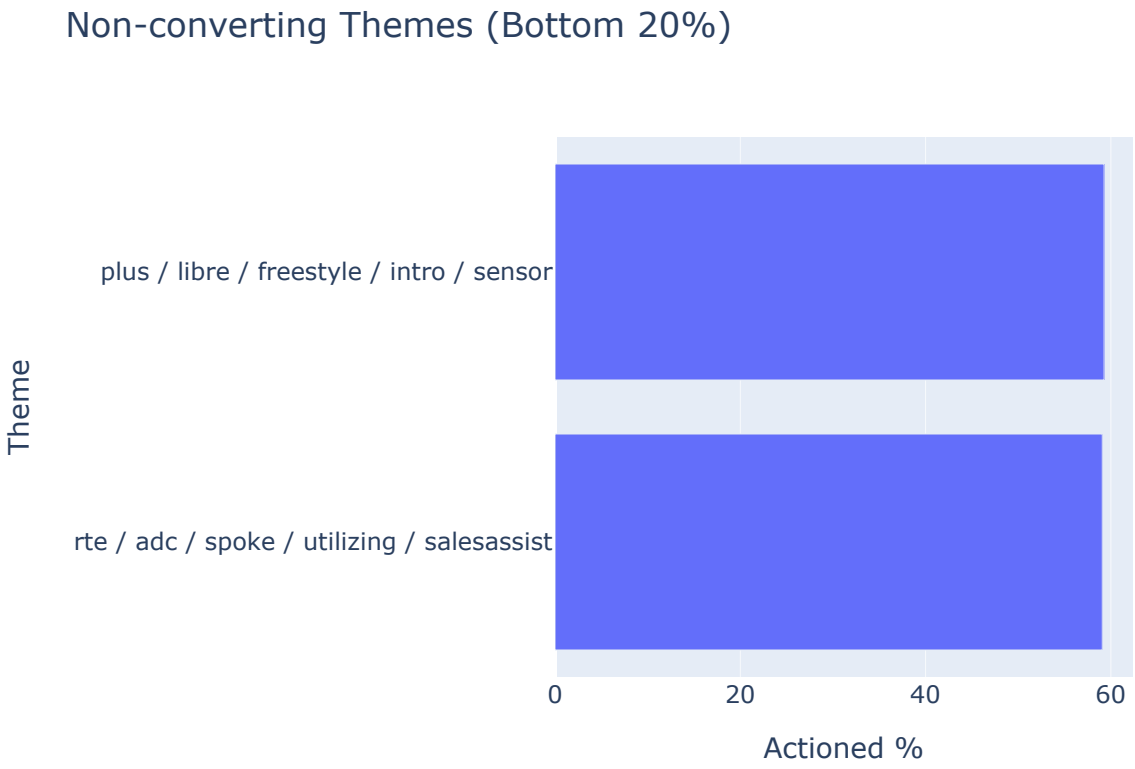
Rep Outlier Themes and Non-Converting Themes

```
import pandas as pd
import numpy as np
from bs4 import BeautifulSoup
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation
import plotly.express as px

# 1) Load & clean
df = pd.read_excel("Suggestions YTD 6-11.xlsx", sheet_name="in")
df['Title'] = df['Title'].astype(str).str.lower().str.strip()
df['reason_text'] = (
    df['Reason'].fillna('')
      .apply(lambda x: BeautifulSoup(x,'html.parser').get_text())
```

```
      .str.lower().str.strip()
)
df['combined_text'] = (
    df['Title'] + " " + df['reason_text']
).str.replace(r'\s+', ' ', regex=True).str.strip()
df = df[df['combined_text']!='']

# 2) Fast LDA topic labeling
cv = CountVectorizer(max_features=2000, max_df=0.5, min_df=10, stop_words='english')
dtm = cv.fit_transform(df['combined_text'])
lda = LatentDirichletAllocation(n_components=10, learning_method='online', max_iter=5,
                                batch_size=128, random_state=42)
lda.fit(dtm)

probs = lda.transform(dtm)
df['Topic'] = probs.argmax(axis=1)
terms = cv.get_feature_names_out()
topic_labels = {
    i: " / ".join(terms[j] for j in comp.argsort()[-5:][::-1])
    for i, comp in enumerate(lda.components_)
}
df['Topic_Label'] = df['Topic'].map(topic_labels)

# 3) Build theme_summary with % metrics
for c in ['Dismissed','Marked As Complete','Actioned']:
    df[c] = pd.to_numeric(df[c].fillna(0), errors='coerce')

df_gap = df.dropna(subset=['Topic_Label']).copy()
theme_summary = (
    df_gap.groupby('Topic_Label')
          .agg(Total=('Topic_Label','size'),
               Completed=('Marked As Complete','sum'),
               Dismissed=('Dismissed','sum'),
               Actioned=('Actioned','sum'))
          .assign(
             pct_completed=lambda d:100*d.Completed/d.Total,
             pct_dismissed=lambda d:100*d.Dismissed/d.Total,
             pct_act=lambda d:100*d.Actioned/d.Total
          )
          .reset_index()
)

# 4) Rep Outlier Themes (Owner: Full Name)
rep_counts = (
    df_gap.groupby(['Owner: Full Name','Topic_Label'])
          .size()
          .reset_index(name='Count')
)
rep_counts['zscore'] = rep_counts.groupby('Topic_Label')['Count'] \
                                 .transform(lambda x:(x-x.mean())/x.std(ddof=0))
outliers = rep_counts[np.abs(rep_counts['zscore'])>2] \
           .sort_values(['Topic_Label','zscore'], ascending=[True, False])
outliers['Owner/Theme'] = outliers['Owner: Full Name'] + " / " + outliers['Topic_Label'
]
```

```
fig1 = px.bar(
    outliers, x='Count', y='Owner/Theme',
    orientation='h',
    title='Rep Outlier Themes (|z| > 2)',
    labels={'Count':'# Suggestions','Owner/Theme':'Owner / Theme'}
)
fig1.update_layout(yaxis={'categoryorder':'total ascending'})
fig1.show()

# 5) Non-converting Themes (bottom 20% by action rate)
threshold = theme_summary['pct_act'].quantile(0.2)
non_conv = theme_summary[theme_summary['pct_act']<=threshold] \
           .sort_values('pct_act', ascending=True)

fig2 = px.bar(
    non_conv, x='pct_act', y='Topic_Label',
    orientation='h',
    title='Non-converting Themes (Bottom 20%)',
    labels={'pct_act':'Actioned %','Topic_Label':'Theme'}
)
fig2.update_layout(yaxis={'categoryorder':'total ascending'})
fig2.show()
```





Use the Outlier chart to leverage rep expertise and surface hidden problems.

Use the Non-converter chart to identify and unblock process bottlenecks, and rescue worthwhile ideas.